

Solution Note

Project: FUME Client Intelligence Platform — GenAI Product Intern mini-case **Stack:** React + Tailwind (frontend) · FastAPI (backend) · Google Gemini `gemini-3.5-flash` via the official `google-genai` SDK

What I built

A working prototype that takes a raw coach-client coaching transcript and turns it into a structured, evidence-grounded client intelligence report, reviewable by a coach before being trusted.

Concretely:

- A **FastAPI backend** with a single endpoint, `POST /analyze`, that accepts `{ "conversation": "<transcript text>" }` and returns a validated JSON report.
- A **two-layer LLM pipeline**: Gemini generates the extraction, Pydantic validates it structurally (evidence required, confidence bounded, classification restricted to a fixed enum), and a bounded retry loop re-prompts the model with a specific correction note if either JSON parsing or schema validation fails.
- A **React + Tailwind dashboard** that renders every finding as a card showing its value, a classification badge (confirmed fact / client-reported / AI inference / missing / conflicting), a confidence indicator, and expandable evidence quotes — plus a sticky Approve / Edit / Reject bar for human review.

Overall architecture

```

React frontend --POST /analyze--> FastAPI (main.py)
                                |
                                v
                                analysis_service.py
(call LLM -> parse JSON -> validate ->
 retry with correction note on failure)
                                |
                                v
                                llm_service.py
(Gemini API call via google-genai SDK,
 code-fence stripping, one internal
 retry on invalid JSON, error handling)
                                |
                                v
                                ClientIntelligenceReport
(Pydantic schema, evidence-enforced)
                                |
                                v
                                200 { success, data } or 502 { detail }
                                |
                                v
React renders cards; coach reviews and
Approves / Edits / Rejects

```

No database, no authentication, and no deployment configuration exist in this project — the assignment explicitly scoped these out, and the prototype respects that scope rather than adding unused infrastructure.

Key assumptions

- The transcript format is a day-segmented, role-labeled conversation (Client / Coach / Accountability Coach), matching the sample provided with the assignment. The prompt and schema are built around this structure, not a generic chat log format.
- A single LLM call per attempt is sufficient — the pipeline does not split extraction and synthesis into separate calls; one prompt asks Gemini to do both the structured extraction and the classification/confidence assignment in one pass, given the prototype's scope and time constraints.
- Coaches (not clients) are the intended reviewers of this report; the UI and language assume a professional reviewing sensitive client data, not the client themselves.
- Approve/Edit/Reject state in the frontend is UI-only for this prototype — there is no database, so review decisions are not persisted and do not survive a page refresh.

Design decisions

- **A single reusable `GradedField` structure** (`value`, `classification`, `confidence`, `evidence`) is used for every metric in the report, rather than a bespoke shape per category. This keeps the frontend to one card component for all metrics and keeps validation logic in one place.
- **Evidence is enforced at the schema level**, not just requested in the prompt — a Pydantic validator rejects any non-`missing_information` field with zero evidence items. This was a deliberate choice to make grounding a structural guarantee rather than something dependent on the model reliably following instructions.
- **Two independent retry layers** (one inside `llm_service.py` for raw invalid JSON, one inside `analysis_service.py` for both JSON and schema failures, with corrective feedback) rather than one — this was added specifically after observing that a single retry-with- correction-note loop wasn't always enough when the underlying issue was output truncation rather than a formatting mistake the model could reason about.
- **No silent fallback on failure**. If the pipeline can't produce a schema-valid report after all retries, the API returns a `502` with a descriptive error rather than a `200` with a best-effort or partially-fabricated report.
- **Deterministic backfill of `report_id/client_id`** — these are generated in code after parsing, not trusted from the LLM, since identifiers aren't analytical findings.

What could go wrong

- **Output truncation from LLM "thinking" tokens** — encountered and fixed during development (see [docs/Failure_Scenarios.md](#), Scenario 2). This is an ongoing risk with reasoning-capable models and large JSON schemas if token budgets aren't managed carefully.
- **Fabricated evidence** — the schema requires a quote to exist, but does not currently verify that the quote is a real substring of the transcript. A model could satisfy validation with a plausible but fabricated quote.
- **Model/provider availability drift** — during development, the originally-selected Gemini model (`gemini-2.5-flash-lite`) was retired for new users mid-project. Model IDs and availability change faster than application code, which is a real operational risk for anything hard-coding a specific model string.

- **Long transcripts** could exceed what a single LLM call can extract reliably in one pass, given the prototype uses one call for the entire transcript rather than per-day or chunked extraction.

Current limitations

- No database — nothing persists between requests; Approve/Edit/Reject decisions are UI state only.
- No authentication — the API is open, matching the assignment's explicit "no auth" scope.
- No deployment configuration — this runs locally via `uvicorn` and `npm run dev`, not as a hosted service.
- No automated grounding check — evidence quotes are required to exist, but not verified against the actual transcript text.
- Single-call extraction — nutrition, exercise, sleep, symptoms, risk flags, etc. are all extracted in one Gemini call rather than split across specialized calls.
- Risk flags are generated by the LLM rather than computed from deterministic, code-defined thresholds — the prompt asks the model to state an explicit rule it used, but nothing in the backend currently re-derives or double-checks that rule against the raw data.

Future improvements

- **Grounding verification:** after parsing, check each evidence quote against the source transcript text (exact or fuzzy substring match) and reject/retry any field whose evidence doesn't verify.
- **Gemini structured output mode** (`response_schema`, not just `response_mime_type`) to constrain generation more tightly at the token level, reducing both truncation and schema-violation risk.
- **Split extraction from synthesis** into two calls — one that only extracts day-referenced facts, one that only synthesizes the narrative/risk-flag/recommendation layer from the validated extraction — so each call has a narrower, more verifiable job.
- **Rule-based risk flag computation:** move flag-triggering logic (e.g. "sleep < 6h on 3+ of last 7 days") into deterministic backend code that consumes the validated extracted fields, with the LLM only responsible for phrasing the flag in natural language.
- **Persist review decisions** (Approve/Edit/Reject) so they survive a refresh and create an audit trail of coach corrections — this data would also be the natural feedback signal for improving the prompt over time.

Production improvements

For an actual multi-coach deployment (explicitly out of scope for this prototype, but worth naming):

- Authentication and per-coach access control.
- A database to store reports, review history, and edit diffs.
- Deployment/CI configuration (containerization, environment management, health checks beyond the current `/health` endpoint).
- Rate limiting and cost controls around LLM calls.
- Structured logging/observability suitable for tracking hallucination rate and retry frequency in aggregate across many coaches, not just per-request logs.

Why Gemini was selected

The prompt and validation architecture were built provider-agnostically from the start, and this project went through three LLM providers during development: Anthropic Claude (initial design and prompt authoring), then Ollama running locally (to remove any per-request API cost), and finally **Google Gemini**, which is the provider in the current implementation.

The move to Gemini specifically was practical: Ollama was timing out consistently in the local development environment, which made iteration unreliable. Gemini was chosen as the replacement because it is a hosted API (no local model weights or GPU dependency), has a usable free tier suitable for an internship assignment, and — via the official `google-genai` SDK — supports the two features this pipeline depends on directly: a `system_instruction` field for the prompt and a `response_mime_type="application/json"` constraint on output format. Because the LLM integration was isolated to a single module (`llm_service.py`) from the beginning, this provider swap required no changes to the prompt, the schema, the validation logic, the FastAPI routes, or the frontend — only the module responsible for making the actual API call.